APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

KTU CST414 — DEEP LEARNING

MODULE 4 HANDWRITTEN LECTURE NOTES

B.Tech Computer Science and Engineering · Scheme 2019

Course Code: CST414	Credits: 3
Course Name: Deep Learning	Lecture Hours: 36
Faculty In-Charge: Jane Rose	Department: Computer Science & Engineering
Design: High-Contrast Printable Notebook	Font: Authentic Chalk & Kalam Ink

Prepared for students and faculty for classroom lectures, print study, and university examination preparation.

COURSE LECTURE INDEX & MAPPING

Day	Topic ID	Lecture Title	Module	CO	Date
#23	4.1	Recurrent Neural Networks: Computational...	M4	CO4	27-Oct-2025
#24	4.2	Encoder-Decoder Sequence-to-Sequence Arc...	M4	CO4	29-Oct-2025
#25	4.3	Deep Recurrent Networks and Recursive Ne...	M4	CO4	31-Oct-2025
#26	4.4	Modern RNNs: Long Short-Term Memory (LST...	M4	CO4	03-Nov-2025
#27	4.5	Gated Recurrent Unit (GRU) and Modern Se...	M4	CO4	05-Nov-2025

LECTURE #23 · MODULE 4 (Topic 4.1)

Date: 27-Oct-2025
Period: Period 2

Title: Recurrent Neural Networks: Computational Graphs and RNN Design

CO: CO4 Bloom's: Understand / Apply

Prerequisites: Backpropagation Algorithm, Sequence modeling intuition, Matrix multiplication, Day 4 concepts
Key Terminology: Recurrent Neural Network (RNN), Hidden State, Computational Graph Unfolding, Backpropagation Through Time (BPTT), Parameter Sharing Across Time, Temporal Receptive Field

Recurrent Equations And Unfolding

A Recurrent Neural Network processes sequential data x^(1), x^(2), ..., x^(tau) by maintaining an internal hidden state memory vector h^(t) that summarizes past history. State Update Equation: h^(t) = tanh(W_hh * h^(t-1) + W_xh * x^(t) + b_h) Output Equation: o^(t) = W_hy * h^(t) + b_y y_hat^(t) = softmax(o^(t)) (or linear / sigmoid for regression) Computational Graph Unfolding: Unfolding (unrolling) maps the recursive loop representation into a directed acyclic computational graph spanning tau discrete time steps. Key insight: The weight matrices W_hh, W_xh, and W_hy are SHARED across all time steps, enabling the network to generalize to arbitrary sequence lengths.

Backpropagation Through Time

Total sequence loss is the sum of losses at each time step: L = sum_{t=1}^tau L^(t). By the multivariable chain rule, the gradient of loss at step tau with respect to hidden state at early step t is a product of Jacobian matrices: del L^(tau) / del h^(t) = (del L^(tau) / del h^(tau)) * product_{j=t+1}^tau (del h^(j) / del h^(j-1)) Each temporal Jacobian is: del h^(j) / del h^(j-1) = W_hh^T * diag(1 - (h^(j))^2) Vanishing / Exploding Problem: If the largest eigenvalue of W_hh is lambda < 1 and tanh' <= 1: The product contracts as (lambda)^T -> 0, causing gradients from distant past steps to vanish completely. The standard RNN cannot bridge long-term dependencies (typically failing beyond 10-15 steps).

Exam Important Points

- Full state update and output equations for simple vanilla RNN.
- Diagram showing recurrent loop representation vs unfolded computational graph.
- Derivation of Backpropagation Through Time (BPTT) chain rule product.
- Why parameter sharing across time is mandatory for variable-length sequence modeling.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Vanilla RNN Hidden State and Output Calculation

Statement: A simple RNN has parameters W_xh = [0.5, 0.2], W_hh = 0.8, b_h = 0.1, W_hy = 1.2, and b_y = -0.3. The activation function for the hidden state is Tanh, and the output neuron is linear. The initial hidden state is h_0 = 0.0. The input sequence has two time steps: x_1 = [1.0, 2.0]^T and x_2 = [0.5, -1.0]^T. Calculate: (a) Hidden state h_1 and output y_1, and (b) Hidden state h_2 and output y_2.

Given: h_0 = 0.0, W_xh = [0.5, 0.2], W_hh = 0.8, b_h = 0.1, W_hy = 1.2, b_y = -0.3.

Governing Formula: h_t = tanh(W_hh * h_{t-1} + W_xh * x_t + b_h); y_t = W_hy * h_t + b_y

Step-by-Step Calculation:

Step 1: Time step t = 1: W_xh * x_1 = (0.5 * 1.0) + (0.2 * 2.0) = 0.5 + 0.4 = 0.9. z_1 = (0.8 * 0.0) + 0.9 + 0.1 = 1.0. h_1 = tanh(1.0) = 0.76159. y_1 = (1.2 * 0.76159) + (-0.3) = 0.9139 - 0.3 = 0.6139.

Step 2: Time step t = 2: W_xh * x_2 = (0.5 * 0.5) + (0.2 * -1.0) = 0.25 - 0.20 = 0.05. W_hh * h_1 = 0.8 * 0.76159 = 0.60927. z_2 = 0.60927 + 0.05 + 0.1 = 0.75927. h_2 = tanh(0.75927) = 0.64082. y_2 = (1.2 * 0.64082) + (-0.3) = 0.7690 - 0.3 = 0.4690.

[FINAL EVALUATED ANSWER: Step 1: h_1 = 0.7616, y_1 = 0.6139. Step 2: h_2 = 0.6408, y_2 = 0.4690.]

Pedagogical Note: The hidden state h_1 retained memory of x_1 and propagated it forward into step 2, combining with x_2 to produce output y_2.

Problem 2 (Advanced (KTU Exam Level)): BPTT Gradient Decay Factor over 10 Time Steps

Statement: In a recurrent neural network, the scalar recurrent weight is W_hh = 0.85. At each time step, the hidden activation derivative is approximately tanh'(z) = 1 - h^2 approx 0.80. (a) Calculate the single-step temporal Jacobian factor gamma = W_hh * tanh'(z), (b) Calculate the total gradient attenuation factor across 10 unrolled time steps: gamma^10, and (c) Determine what percentage of the gradient signal survives backpropagating 10 steps into the past.

Given: W_hh = 0.85, tanh'(z) = 0.80, horizon T = 10 steps.

Governing Formula: Single step gamma = W_hh * tanh'(z); 10-step attenuation = gamma^10; Survival % = gamma^10 * 100

Step-by-Step Calculation:

Step 1: Calculate single-step contraction factor: gamma = 0.85 * 0.80 = 0.680.

Step 2: Calculate attenuation across 10 steps: Attenuation = (0.680)^10 = 0.021139.

Step 3: Calculate percentage of surviving gradient: Percentage = 0.021139 * 100 = 2.11%.

[FINAL EVALUATED ANSWER: Single-step factor gamma = 0.680. 10-step attenuation = 0.0211 (Only 2.11% gradient survives).]

Pedagogical Note: Over 97.8% of the error gradient disappears after just 10 time steps, demonstrating why vanilla RNNs cannot learn long-term temporal dependencies.

Real-World Industry Case (Renewable Energy & Power Distribution): Smart Power Grid Wind Energy Output Forecasting

Pipeline Workflow: Hourly anemometer wind speed and barometric sensor readings over past 24 hours -> Unrolled RNN processes sequential wind gusts -> Predicts mega-watt turbine power generation for next 4 hours.

Industrial Impact & Benefit: Enables power grid operators to balance battery storage and spin up backup generation without blackouts.

Key Oral Viva Voce Checkpoints

Q1: What is the primary architectural difference between an MLP and an RNN?

Ans: An RNN contains cyclic recurrent connections that feed the previous hidden state back into the current step, allowing it to process variable-length sequential data with memory.

Q2: What does 'unfolding' a recurrent computational graph mean?

Ans: Drawing the computational graph across discrete time steps as a sequence of repeated identical layers sharing the same parameters.

Q3: What is Backpropagation Through Time (BPTT)?

Ans: The application of the backpropagation algorithm to an unfolded recurrent network, computing gradients by summing contributions across all time steps.

Q4: Why is parameter sharing essential in RNNs?

Ans: It allows the model to process sequences of arbitrary length and recognize patterns regardless of where they appear in time, using a fixed number of parameters.

Q5: Why do vanilla RNNs suffer from vanishing gradients?

Ans: Because backpropagation over long horizons involves repeatedly multiplying by the recurrent weight matrix W_hh and tanh derivatives (< 1), causing gradients to decay exponentially.

LECTURE #24 · MODULE 4 (Topic 4.2)

Date: 29-Oct-2025
Period: Period 4

Title: Encoder-Decoder Sequence-to-Sequence Architectures

CO: CO4 Bloom's: Understand / Apply

Prerequisites: Vanilla RNN, Hidden states, Day 23 concepts
Key Terminology: Sequence-to-Sequence (Seq2Seq), Encoder RNN, Decoder RNN, Context Vector, Information Bottleneck, Teacher Forcing, Autoregressive Generation, Beam Search

Seq2Seq Architecture

Standard RNNs require input length T_x and output length T_y to be strictly equal or synchronized. Many tasks require mapping a sequence of length T_x to an output sequence of different length T_y (e.g. Machine Translation: 'How are you?' [3 words] -> 'Wie geht es Ihnen?' [4 words]). 1. Encoder RNN: Processes input sequence x^(1), ..., x^(T_x) sequentially: h_enc^(t) = f_enc(h_enc^(t-1), x^(t)) The final hidden state represents the summary Context Vector c = h_enc^(T_x). 2. Decoder RNN: Initializes hidden state with the context vector: h_dec^(0) = c. Autoregressively generates target tokens y^(1), ..., y^(T_y): h_dec^(t) = f_dec(h_dec^(t-1), y^(t-1), c) p(y^(t) | y^(1), ..., y^(t-1), x) = softmax(W_out * h_dec^(t)) Generation terminates when the special <EOS> (End-of-Sequence) token is emitted.

Information Bottleneck And Teacher Forcing

1. Information Bottleneck Problem: Compressing an entire source sentence of arbitrary length (e.g. 50 words) into a single fixed-dimensional vector c (e.g. 512 floats) creates an extreme bottleneck. Model performance deteriorates sharply for sentences longer than 20-30 words (motivating Bahdanau's Attention mechanism). 2. Teacher Forcing: During training, the decoder receives the ground-truth target token y_true^(t-1) as input at step t, rather than its own potentially erroneous previous prediction y_pred^(t-1). - Advantage: Stabilizes and drastically accelerates training. - Drawback (Exposure Bias): At test time, ground truth is absent; early mistakes can compound catastrophically.

Exam Important Points

- Full architectural diagram of Encoder-Decoder Seq2Seq framework.
- How context vector c transfers semantic information between encoder and decoder.
- Definition, merits, and exposure bias drawback of Teacher Forcing.
- Why fixed-length context vectors fail on long sequences.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Context Vector Compression Ratio in Seq2Seq

Statement: An English sentence of 40 words is processed by an Encoder RNN. Each word is represented by a 300-dimensional word embedding vector. The Encoder condenses the sentence into a context vector c of dimension 512. Assuming 32-bit floating point numbers (4 bytes/float): (a) Calculate the raw input memory size in bytes, (b) Calculate the context vector memory size in bytes, and (c) Determine the compression ratio achieved by the bottleneck.

Given: 40 words, 300 dims/word, context c = 512 floats, 4 bytes/float.

Governing Formula: Input_mem = Words * Dims * 4; Context_mem = Dims_c * 4; Ratio = Input_mem / Context_mem

Step-by-Step Calculation:

Step 1: Compute input sentence memory: Total input floats = 40 * 300 = 12,000 floats. Input memory = 12,000 * 4 bytes = 48,000 bytes = 46.88 KB.

Step 2: Compute context vector memory: Context memory = 512 * 4 bytes = 2,048 bytes = 2.0 KB.

Step 3: Calculate compression ratio: Ratio = 48,000 / 2,048 = 23.4375x (over 23-fold compression!).

[FINAL EVALUATED ANSWER: Input = 48,000 bytes. Context = 2,048 bytes. Compression ratio = 23.44x.]

Pedagogical Note: The encoder must compress 48 KB of nuanced semantic and syntactic linguistic relationships into just 2 KB, highlighting why fixed context vectors suffer severe information loss on long documents.

Problem 2 (Advanced (KTU Exam Level)): Beam Search Path Probability vs Greedy Decoding

Statement: At step 1 of decoding, a vocabulary has 3 words: {A, B, C} with probabilities P(A) = 0.6, P(B) = 0.3, P(C) = 0.1. At step 2, conditional probabilities are: If step 1 was A: P(A|A) = 0.2, P(B|A) = 0.3, P(C|A) = 0.5. If step 1 was B: P(A|B) = 0.8, P(B|B) = 0.1, P(C|B) = 0.1. (a) Determine the 2-step sequence selected by Greedy Decoding, (b) Calculate the total sequence probability of all 9 possible combinations, and (c) Determine the optimal sequence found by Beam Search with beam width B = 2.

Given: P(y1), P(y2|y1) as specified.

Governing Formula: P(y1, y2) = P(y1) * P(y2|y1); Greedy: argmax at each step; Beam Search: maintain top B paths at each step

Step-by-Step Calculation:

Step 1: Greedy Decoding: Step 1: max(0.6, 0.3, 0.1) -> selects 'A' (P=0.6). Step 2 (given A): max(0.2, 0.3, 0.5) -> selects 'C' (P=0.5). Greedy sequence: 'A C' with total probability = 0.6 * 0.5 = 0.30.

Step 2: Evaluate alternative candidate paths starting from B: P('B A') = P(B) * P(A|B) = 0.3 * 0.8 = 0.24. P('B B') = 0.3 * 0.1 = 0.03. P('B C') = 0.3 * 0.1 = 0.03.

Step 3: Evaluate paths starting from A: P('A A') = 0.6 * 0.2 = 0.12. P('A B') = 0.6 * 0.3 = 0.18. P('A C') = 0.6 * 0.5 = 0.30.

Step 4: Beam Search with Beam Width B = 2: Step 1: Top 2 candidates retained: Path 'A' (0.6) and Path 'B' (0.3). Step 2 expansions: 'A A' = 0.12, 'A B' = 0.18, 'A C' = 0.30, 'B A' = 0.24, 'B B' = 0.03, 'B C' = 0.03. Top 2 pruned beam paths: 'A C' (0.30) and 'B A' (0.24). Optimal path = 'A C' (0.30).

[FINAL EVALUATED ANSWER: Greedy = 'A C' (P = 0.30). Beam Search (B=2) retains both 'A C' (0.30) and 'B A' (0.24), confirming 'A C' as optimal.]

Pedagogical Note: Greedy search makes locally optimal decisions that can miss globally superior sequences, whereas Beam Search explores multiple plausible trajectories concurrently.

Real-World Industry Case (Medical Informatics & Cross-Lingual Healthcare): Automated Clinical Discharge Summary Translation

Pipeline Workflow: Doctor dictates English clinical notes -> Encoder RNN ingests medical diagnosis jargon -> Produces latent context embedding -> Decoder RNN generates patient-friendly regional language discharge instructions.

Industrial Impact & Benefit: Eliminates language barriers in rural hospital discharge, cutting medication dosing misunderstandings by 35%.

Key Oral Viva Voce Checkpoints

Q1: What is the core capability that Seq2Seq provides over standard vanilla RNNs?

Ans: Mapping an input sequence of length T_x to an output sequence of a completely different length T_y.

Q2: What is the Context Vector c in Seq2Seq?

Ans: The final hidden state vector of the Encoder RNN that summarizes the entire source sentence and serves as the seed memory for the Decoder.

Q3: What is the Information Bottleneck problem in Seq2Seq?

Ans: Forcing an entire source sentence of arbitrary length into a single fixed-size vector causes severe performance degradation on long sequences.

Q4: What is Teacher Forcing?

Ans: A training procedure where the ground-truth target token from the previous step is fed into the decoder, rather than the model's own predicted token.

Q5: What is Exposure Bias in sequence generation?

Ans: The discrepancy between training (where ground truth tokens are provided) and inference (where the model must consume its own predictions), allowing early generation errors to compound.

LECTURE #25 · MODULE 4 (Topic 4.3)

Date: 31-Oct-2025
Period: Period 1

Title: Deep Recurrent Networks and Recursive Neural Networks

CO: CO4 Bloom's: Understand / Apply

Prerequisites: RNN Unfolding, Seq2Seq, Tree data structures, Day 24 concepts
Key Terminology: Deep Recurrent Network, Stacked RNN, Bidirectional RNN (BiRNN), Forward State, Backward State, Recursive Neural Network, Tree-Structured Graph, Constituency Parse Tree

Deep Stacked Rnns

A standard RNN is deep in time (across unrolled sequence steps), but shallow in spatial representation (only one hidden layer per time step). A Deep (Stacked) RNN stacks multiple recurrent layers vertically: - Layer 1 receives input sequence x^(t): h_1^(t) = f(W_1 * x^(t) + U_1 * h_1^(t-1)) - Layer 2 receives hidden state sequence of Layer 1 as its input: h_2^(t) = f(W_2 * h_1^(t) + U_2 * h_2^(t-1)) - Layer L computes: h_L^(t) = f(W_L * h_{L-1}^(t) + U_L * h_L^(t-1)) Hierarchical Temporal Abstraction: Lower recurrent layers capture fast, fine-grained temporal patterns (phonemes, characters), while higher recurrent layers capture slow, abstract semantic structures (syntax, topics).

Bidirectional Rnns

Schuster & Paliwal (1997) introduced Bidirectional RNNs (BiRNNs) to resolve the limitation that standard causal RNNs only know past context. Equations: 1. Forward RNN: h_fwd^(t) = f(W_xf * x^(t) + W_ff * h_fwd^(t-1)) (processes sequence left-to-right) 2. Backward RNN: h_bwd^(t) = f(W_xb * x^(t) + W_bb * h_bwd^(t+1)) (processes sequence right-to-left) 3. Concatenated Representation: h^(t) = [h_fwd^(t) ; h_bwd^(t)] Output y^(t) is conditioned simultaneously on past history AND future context! Critical for speech recognition, POS tagging, and NER.

Recursive Neural Networks

While an RNN operates over a 1D chain graph of sequence length T, a Recursive Neural Network (Pollack 1990; Socher et al. 2011) operates over an arbitrary Directed Tree-Structured Graph. Tree Propagation: For a node with children c_1 and c_2: p = f(W * [c_1 ; c_2] + b) The parent representation p is computed recursively up to the root node. Advantage: Drastically reduces the maximum path length between any two words from O(T) in a linear RNN to O(log T) in a balanced tree, massively mitigating vanishing gradients and naturally matching the hierarchical syntax of natural language sentences and molecular graphs.

Exam Important Points

- Architectural equations and diagram for Bidirectional RNNs.
- Comparison: Recurrent (1D sequential chain) vs Recursive (hierarchical tree graph).
- Why BiRNNs cannot be used for online real-time causal generation.
- Logarithmic path length O(log T) in Recursive networks.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Trainable Parameter Count in a 3-Layer Deep Stacked RNN

Statement: A deep stacked RNN has an input dimension of x_dim = 128. It contains 3 stacked recurrent hidden layers, each having hidden dimension h_dim = 256. The top hidden layer connects to an output layer of dimension y_dim = 10. Assuming each layer has full input-to-hidden and hidden-to-hidden matrices plus biases: (a) Calculate the parameters in Layer 1, (b) Calculate the parameters in Layer 2 and Layer 3, (c) Calculate output layer parameters, and (d) Determine the total trainable parameters.

Given: x_dim = 128, h1 = 256, h2 = 256, h3 = 256, y_dim = 10.

Governing Formula: Layer 1 = (x_dim * h) + (h * h) + h; Middle Layer = (h_prev * h) + (h * h) + h; Out = (h * y) + y

Step-by-Step Calculation:

Step 1: Layer 1 (Input 128 -> Hidden 256): W_x1 = 256 * 128 = 32,768. W_h1 = 256 * 256 = 65,536. b_1 = 256. Total Layer 1 = 32,768 + 65,536 + 256 = 98,560 parameters.

Step 2: Layer 2 (Hidden 256 -> Hidden 256): W_x2 = 256 * 256 = 65,536. W_h2 = 256 * 256 = 65,536. b_2 = 256. Total Layer 2 = 65,536 + 65,536 + 256 = 131,328 parameters.

Step 3: Layer 3 (Hidden 256 -> Hidden 256): Total Layer 3 = 131,328 parameters.

Step 4: Output Layer (Hidden 256 -> Output 10): W_out = 10 * 256 = 2,560. b_out = 10. Total Output = 2,560 + 10 = 2,570 parameters.

Step 5: Total Trainable Parameters = 98,560 + 131,328 + 131,328 + 2,570 = 363,786 parameters.

[FINAL EVALUATED ANSWER: Layer 1 = 98,560; Layer 2 = 131,328; Layer 3 = 131,328; Output = 2,570. Total = 363,786 parameters.]

Pedagogical Note: Stacking layers creates multi-tier hierarchical representations with moderate parameter growth compared to widening.

Problem 2 (Advanced (KTU Exam Level)): Path Length and Gradient Propagation: Recurrent Chain vs Recursive Tree

Statement: Consider a sequence of length T = 64 tokens. (a) In a standard 1D recurrent chain, calculate the maximum number of sequential transitions required for an error signal to propagate from token 64 to token 1, (b) In a balanced binary recursive parse tree, calculate the maximum path length from the root to any leaf token, and (c) Assuming a per-step contraction factor gamma = 0.8, compare the gradient attenuation in the Recurrent chain vs the Recursive tree.

Given: T = 64 tokens, gamma = 0.8.

Governing Formula: Recurrent Path = T - 1; Recursive Balanced Tree Path = 2 * log2(T); Attenuation = gamma^(Path)

Step-by-Step Calculation:

Step 1: Recurrent 1D Chain Path Length: Path = 64 - 1 = 63 transitions.

Step 2: Recursive Balanced Binary Tree Path Length: Tree height = log2(64) = 6 levels. Maximum path between two leaves through root = 2 * 6 = 12 transitions.

Step 3: Compute Recurrent gradient attenuation: Attenuation_RNN = (0.8)^63 = 1.07 x 10^(-6) (Completely vanished!).

Step 4: Compute Recursive gradient attenuation: Attenuation_Recursive = (0.8)^12 = 0.0687 (6.87% signal preserved!).

Step 5: Comparison ratio: Ratio = 0.0687 / (1.07 x 10^(-6)) = ~64,000x stronger gradient signal!

[FINAL EVALUATED ANSWER: Recurrent Path = 63 steps (Atten = 1.07e-6). Recursive Tree Path = 12 steps (Atten = 0.0687, ~64,000x stronger signal).]

Pedagogical Note: The tree topology of Recursive Neural Networks compresses sequential path lengths from O(T) to O(log T), dramatically preserving gradient signals across distant words.

Real-World Industry Case (Clinical NLP & Biomedical Information Extraction): Named Entity Recognition (NER) in Electronic Health Records (EHR)

Pipeline Workflow: Clinical doctor discharge note -> Ingested by a 2-layer Bidirectional LSTM -> Forward state understands preceding symptoms, Backward state understands succeeding medication dosages -> Accurately tags entities (Disease, Drug, Dosage).

Industrial Impact & Benefit: Extracts structured clinical adverse drug event records from unstructured physician notes with 93.8% F1-score.

Key Oral Viva Voce Checkpoints

Q1: What is the primary motivation for Bidirectional RNNs?

Ans: Standard RNNs only possess information about past context; BiRNNs process the sequence in both forward and backward directions, giving each token full access to both past and future context.

Q2: Why can Bidirectional RNNs NOT be used for real-time online speech generation or autoregressive language modeling?

Ans: Because the backward pass requires access to future tokens, which have not yet occurred or been spoken in real time.

Q3: What is a Deep (Stacked) RNN?

Ans: An architecture where the output hidden states of one recurrent layer serve as the input sequence to a second recurrent layer above it.

Q4: What is the primary difference between a Recurrent and a Recursive Neural Network?

Ans: A Recurrent network operates over a 1D linear chain of time steps, whereas a Recursive network operates over an arbitrary hierarchical tree graph (e.g. syntax parse tree).

Q5: What is the theoretical advantage of a Recursive Neural Network for natural language?

Ans: It matches the innate hierarchical constituency grammar of human language and reduces the gradient propagation distance between words from O(T) to O(log T).

LECTURE #26 · MODULE 4 (Topic 4.4)

Date: 03-Nov-2025
Period: Period 2

Title: Modern RNNs: Long Short-Term Memory (LSTM)

CO: CO4 Bloom's: Understand / Apply

Prerequisites: Vanilla RNN, Backpropagation Through Time, Vanishing Gradients, Day 23 concepts
Key Terminology: Long Short-Term Memory (LSTM), Cell State, Constant Error Carousel (CEC), Forget Gate, Input Gate, Output Gate, Candidate Memory State, Additive Gradient Highway

Lstm Architecture And Equations

Hochreiter & Schmidhuber (1997) designed the LSTM to maintain a linear memory cell state c^(t) where error gradients can flow backward across time without exponential decay. Given input x^(t) and previous hidden state h^(t-1): 1. Forget Gate (Gers et al., 2000): f^(t) = sigma(W_f * [h^(t-1) ; x^(t)] + b_f) Decides what fraction of the old cell state to discard (0 = forget completely, 1 = retain entirely). 2. Input Gate & Candidate State: i^(t) = sigma(W_i * [h^(t-1) ; x^(t)] + b_i) (determines which values to update) c_tilde^(t) = tanh(W_c * [h^(t-1) ; x^(t)] + b_c) (creates new candidate values) 3. Cell State Update (The Additive Highway): c^(t) = f^(t) odot c^(t-1) + i^(t) odot c_tilde^(t) Key Insight: This is a LINEAR, ADDITIVE combination! If the forget gate f^(t) = 1, the old memory passes through with a derivative of exactly 1.0, enabling error signals to flow back hundreds of steps without decaying. 4. Output Gate & Hidden State: o^(t) = sigma(W_o * [h^(t-1) ; x^(t)] + b_o) h^(t) = o^(t) odot tanh(c^(t)) Filters the cell state through the output gate to produce the external hidden state.

Mathematical Resolution Of Vanishing Gradients

In a vanilla RNN, del h^(t) / del h^(t-1) = W_hh^T * diag(1 - h^2). Repeated multiplication of matrices with eigenvalues < 1 causes exponential decay. In an LSTM, the gradient flowing along the cell state is: del c^(t) / del c^(t-1) = diag(f^(t)) If the network learns that forget gate f^(t) approx 1.0, the Jacobian is the IDENTITY matrix: del c^(tau) / del c^(t) = product_{j=t+1}^tau f^(j) approx 1.0 Gradients bypass the destructive multiplication by weight matrices entirely, flowing uninterrupted across hundreds of steps.

Exam Important Points

- Full mathematical formulas for all four LSTM gates/states: f, i, c_tilde, c, o, h.
- Architectural block diagram showing cell state conveyor belt and pointwise operations.
- Mathematical explanation of why del c^(t) / del c^(t-1) = f^(t) resolves vanishing gradients.
- KTU Exam Q18(a): Applications of LSTM in Natural Language Processing (Machine Translation, Text Generation, Sentiment Analysis).

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Manual Step-by-Step LSTM Gate and State Calculation

Statement: An LSTM has scalar dimensions (x_dim = 1, h_dim = 1). At step t = 1, previous hidden state is h_0 = 0.5 and previous cell state is c_0 = 2.0. The input is x_1 = 1.0. Parameters are: Forget gate: W_f = 0.5, U_f = 0.2, b_f = 0.1 (Sigmoid) Input gate: W_i = 0.4, U_i = 0.3, b_i = -0.2 (Sigmoid) Candidate state: W_c = 0.8, U_c = 0.1, b_c = 0.0 (Tanh) Output gate: W_o = 0.6, U_o = -0.1, b_o = 0.2 (Sigmoid) Calculate: (a) Forget gate f_1, (b) Input gate i_1 and candidate c_tilde_1, (c) Updated cell state c_1, and (d) Updated hidden state h_1.

Given: h_0 = 0.5, c_0 = 2.0, x_1 = 1.0. Parameters as stated.

Governing Formula: f = sigma(W_f*x + U_f*h + b_f); i = sigma(W_i*x + U_i*h + b_i); c_tilde = tanh(W_c*x + U_c*h + b_c); c = f*c_prev + i*c_tilde; o = sigma(W_o*x + U_o*h + b_o); h = o * tanh(c)

Step-by-Step Calculation:

Step 1: Compute Forget Gate f_1: z_f = (0.5 * 1.0) + (0.2 * 0.5) + 0.1 = 0.5 + 0.1 + 0.1 = 0.70. f_1 = 1 / (1 + e^(-0.70)) = 1 / (1 + 0.49659) = 1 / 1.49659 = 0.6682.

Step 2: Compute Input Gate i_1 and Candidate c_tilde_1: z_i = (0.4 * 1.0) + (0.3 * 0.5) + (-0.2) = 0.4 + 0.15 - 0.2 = 0.35. i_1 = 1 / (1 + e^(-0.35)) = 1 / (1 + 0.70469) = 1 / 1.70469 = 0.5866. z_c = (0.8 * 1.0) + (0.1 * 0.5) + 0.0 = 0.8 + 0.05 = 0.85. c_tilde_1 = tanh(0.85) = 0.6911.

Step 3: Update Cell State c_1: c_1 = (f_1 * c_0) + (i_1 * c_tilde_1) = (0.6682 * 2.0) + (0.5866 * 0.6911) = 1.3364 + 0.4054 = 1.7418.

Step 4: Compute Output Gate o_1 and Hidden State h_1: z_o = (0.6 * 1.0) + (-0.1 * 0.5) + 0.2 = 0.6 - 0.05 + 0.2 = 0.75. o_1 = 1 / (1 + e^(-0.75)) = 1 / (1 + 0.47237) = 1 / 1.47237 = 0.6792. tanh(c_1) = tanh(1.7418) = 0.9404. h_1 = o_1 * tanh(c_1) = 0.6792 * 0.9404 = 0.6387.

[FINAL EVALUATED ANSWER: f_1 = 0.6682, i_1 = 0.5866, c_tilde_1 = 0.6911, Cell State c_1 = 1.7418, Hidden State h_1 = 0.6387]

Pedagogical Note: The forget gate retained 66.8% of past memory c_0, added new information scaled by i_1, and exposed 63.8% of the transformed memory to the hidden state.

Problem 2 (Advanced (KTU Exam Level)): Trainable Parameter Count in Standard Multi-Class LSTM Layer

Statement: An LSTM layer receives sequential word embeddings of dimension x_dim = 300 and contains h_dim = 512 hidden units. It connects to an output classification vocabulary of size V = 10,000 words. (a) Derive the parameter formula for an LSTM layer containing 4 internal gates/states, (b) Calculate total weights and biases in the recurrent LSTM layer, (c) Calculate parameters in the final projection layer, and (d) Determine the grand total trainable parameters.

Given: x_dim = 300, h_dim = 512, V = 10,000.

Governing Formula: LSTM Params = 4 * [h_dim * (x_dim + h_dim) + h_dim]; Projection = (h_dim * V) + V

Step-by-Step Calculation:

Step 1: Understand LSTM 4-fold parameter structure: The LSTM contains 4 linear transformations: Forget gate, Input gate, Candidate cell, Output gate. Each gate has input weights (h x x_dim), recurrent weights (h x h), and bias (h).

Step 2: Parameters per single gate: W_x = 512 * 300 = 153,600. W_h = 512 * 512 = 262,144. b = 512. Per-gate parameters = 153,600 + 262,144 + 512 = 416,256.

Step 3: Total LSTM Layer parameters (4 gates): LSTM_total = 4 * 416,256 = 1,665,024 parameters.

Step 4: Output Projection Layer: W_proj = 10,000 * 512 = 5,120,000. b_proj = 10,000. Total Projection = 5,120,000 + 10,000 = 5,130,000 parameters.

Step 5: Grand Total Parameters = 1,665,024 + 5,130,000 = 6,795,024 parameters.

[FINAL EVALUATED ANSWER: LSTM Layer = 1,665,024 params. Output Layer = 5,130,000 params. Grand Total = 6,795,024 parameters.]

Pedagogical Note: An LSTM has exactly 4 times the recurrent parameters of a vanilla RNN of the same hidden width due to its four internal gating projections.

Real-World Industry Case (Critical Care Medicine & Early Warning Clinical Systems): Long-Term ICU Patient Sepsis Onset Prediction

Pipeline Workflow: Patient vital signs recorded every minute over a 72-hour ICU stay (4,320 time steps) -> LSTM cell state conveyor belt maintains long-term baseline organ history -> Predicts septic shock 6 hours before clinical manifestation.

Industrial Impact & Benefit: Enables preemptive broad-spectrum antibiotic intervention, reducing ICU sepsis mortality by 28%.

Key Oral Viva Voce Checkpoints

Q1: Who invented the LSTM architecture and in what year?
Ans: Sepp Hochreiter and Jürgen Schmidhuber in 1997.

Q2: What are the three gates present in a standard modern LSTM?
Ans: The Forget Gate, the Input Gate, and the Output Gate.

Q3: What is the mathematical role of the Forget Gate?
Ans: To compute a probability vector f in [0, 1] using a Sigmoid function that determines which components of the previous cell state c^(t-1) are preserved or erased.

Q4: Why is the cell state update c^(t) = f*c^(t-1) + i*c_tilde called an additive conveyor belt?
Ans: Because information is added linearly without passing through a matrix multiplication, allowing error gradients to propagate unchanged with derivative ~ 1.0.

Q5: Why should the forget gate bias b_f be initialized to 1.0 or 2.0 at the start of training?
Ans: To ensure that f^(t) approx 1.0 initially, preventing the network from forgetting historical context during early training epochs.

LECTURE #27 · MODULE 4 (Topic 4.5)

Date: 05-Nov-2025
Period: Period 4

Title: Gated Recurrent Unit (GRU) and Modern Sequence Architectures

CO: CO4 Bloom's: Understand / Apply

Prerequisites: LSTM Architecture, Gating mechanisms, Day 26 concepts
Key Terminology: Gated Recurrent Unit (GRU), Reset Gate, Update Gate, Merged Hidden State, Convex Linear Interpolation, Parameter Efficiency, LSTM vs GRU Trade-off

Gru Architecture And Equations

Kyunghyun Cho et al. (2014) introduced the Gated Recurrent Unit (GRU) as a streamlined, computationally efficient alternative to LSTM. Key Structural Differences from LSTM: 1. No separate Cell State: GRU merges cell state c^(t) and hidden state h^(t) into a single state vector h^(t). 2. Two Gates instead of Three: Eliminates the separate output gate. Uses only a Reset Gate r^(t) and an Update Gate z^(t). 3. Coupled Retention and Injection: In LSTM, forget gate f and input gate i are independent. In GRU, the update gate z controls both simultaneously via convex linear interpolation (1 - z) and z. Mathematical Formulation: 1. Update Gate: z^(t) = sigma(W_z * [h^(t-1) ; x^(t)] + b_z) Decides how much of past state to carry forward vs new state. 2. Reset Gate: r^(t) = sigma(W_r * [h^(t-1) ; x^(t)] + b_r) Decides how much of past state to ignore when computing candidate. 3. Candidate Hidden State: h_tilde^(t) = tanh(W_h * [r^(t) odot h^(t-1) ; x^(t)] + b_h) 4. Final Hidden State (Linear Interpolation): h^(t) = (1 - z^(t)) odot h^(t-1) + z^(t) odot h_tilde^(t)

Lstm Vs Gru Comprehensive Comparison

Comparison Dimension | LSTM | GRU Gates | 3 (Forget, Input, Output) | 2 (Reset, Update) Internal States | 2 (Cell c + Hidden h) | 1 (Merged Hidden h) Parameters per Unit | 4 * (d_x + d_h + 1) * d_h | 3 * (d_x + d_h + 1) * d_h (25% fewer parameters!) Computation Speed | Slower (more matrix multiplies) | Faster (~25-30% faster per epoch) Memory Usage | Higher (stores c and h) | Lower Long Dependencies | Superior on massive data / long sequences | Comparable on moderate sequences, less prone to overfitting on small datasets.

Exam Important Points

- KTU Exam Q18(b): Complete architectural discussion and equations of GRU.
- Equations for Update Gate z, Reset Gate r, Candidate h_tilde, and Final h.
- How GRU couples forgetting and updating via (1 - z)*h_prev + z*h_tilde.
- Detailed comparison table: LSTM vs GRU.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Manual Step-by-Step GRU State Calculation

Statement: A GRU cell has scalar dimensions (x_dim = 1, h_dim = 1). At step t = 1, the previous hidden state is h_0 = 0.6. The input is x_1 = 0.5. Parameters are: Reset gate: W_xr = 0.4, W_hr = 0.5, b_r = -0.1 (Sigmoid) Update gate: W_xz = 0.3, W_hz = 0.2, b_z = 0.1 (Sigmoid) Candidate state: W_xh = 0.6, W_hh = 0.8, b_h = 0.0 (Tanh) Calculate: (a) Reset gate r_1, (b) Update gate z_1, (c) Candidate state h_tilde_1, and (d) Final updated hidden state h_1.

Given: h_0 = 0.6, x_1 = 0.5. Parameters as specified.

Governing Formula: r = sigma(W_xr*x + W_hr*h + b_r); z = sigma(W_xz*x + W_hz*h + b_z); h_tilde = tanh(W_xh*x + W_hh*(r*h) + b_h); h = (1 - z)*h_prev + z*h_tilde

Step-by-Step Calculation:

Step 1: Compute Reset Gate r_1: z_r = (0.4 * 0.5) + (0.5 * 0.6) + (-0.1) = 0.20 + 0.30 - 0.10 = 0.40. r_1 = 1 / (1 + e^(-0.40)) = 1 / (1 + 0.67032) = 1 / 1.67032 = 0.5987.

Step 2: Compute Update Gate z_1: z_z = (0.3 * 0.5) + (0.2 * 0.6) + 0.10 = 0.15 + 0.12 + 0.10 = 0.37. z_1 = 1 / (1 + e^(-0.37)) = 1 / (1 + 0.69073) = 1 / 1.69073 = 0.5915.

Step 3: Compute Candidate State h_tilde_1: Reset-masked previous state = r_1 * h_0 = 0.5987 * 0.6 = 0.3592. z_h = (0.6 * 0.5) + (0.8 * 0.3592) + 0.0 = 0.30 + 0.2874 = 0.5874. h_tilde_1 = tanh(0.5874) = 0.5281.

Step 4: Compute Final Hidden State h_1: h_1 = (1.0 - z_1) * h_0 + (z_1 * h_tilde_1) = (1.0 - 0.5915) * 0.6 + (0.5915 * 0.5281) = (0.4085 * 0.6) + 0.3124 = 0.2451 + 0.3124 = 0.5575.

[FINAL EVALUATED ANSWER: Reset gate r_1 = 0.5987, Update gate z_1 = 0.5915, Candidate h_tilde_1 = 0.5281, Final Hidden State h_1 = 0.5575]

Pedagogical Note: The update gate z_1 = 0.5915 blended 40.85% of old state h_0 with 59.15% of the newly computed candidate h_tilde_1.

Problem 2 (Advanced (KTU Exam Level)): Exact Parameter Reduction Ratio: GRU vs LSTM

Statement: An embedded edge speech recognition system has an input dimension of x_dim = 80 (filterbank features) and a recurrent hidden dimension of h_dim = 256. (a) Calculate the total weights and biases in an LSTM layer, (b) Calculate the total weights and biases in a GRU layer, (c) Determine the exact number of parameters saved and the percentage reduction, and (d) Calculate the GPU inference memory saved for a batch size of 64 over 100 time steps.

Given: x_dim = 80, h_dim = 256, batch = 64, T = 100.

Governing Formula: LSTM = 4 * [h*(x + h) + h]; GRU = 3 * [h*(x + h) + h]; Ratio = 3 / 4 = 0.75 (25% reduction)

Step-by-Step Calculation:

Step 1: Compute single-gate parameters: Single gate = 256 * (80 + 256) + 256 = 256 * 336 + 256 = 86,016 + 256 = 86,272 parameters.

Step 2: LSTM Parameters (4 gates): LSTM_total = 4 * 86,272 = 345,088 parameters.

Step 3: GRU Parameters (3 gates): GRU_total = 3 * 86,272 = 258,816 parameters.

Step 4: Compute parameter reduction: Difference = 345,088 - 258,816 = 86,272 parameters. Percentage Reduction = (86,272 / 345,088) * 100 = 25.00% exact reduction!

Step 5: Compute state cache memory savings at inference: LSTM caches both cell state c and hidden state h: 2 * (64 * 256) floats = 32,768 floats = 128 KB. GRU caches only hidden state h: 1 * (64 * 256) floats = 16,384 floats = 64 KB (50% state memory savings!).

[FINAL EVALUATED ANSWER: LSTM = 345,088 params. GRU = 258,816 params (Exact 25% parameter reduction; 50% state cache savings).]

Pedagogical Note: GRU slashes parameter count by exactly 25% and halves state memory requirements, making it ideal for edge microcontrollers and low-latency audio devices.

Real-World Industry Case (Smart Home IoT & Ultra-Low-Power Edge Audio): On-Device Voice Assistant Wake-Word Detection

Pipeline Workflow: Always-on microphone samples ambient audio at 16 kHz -> 2-layer GRU consumes streaming 80-band Mel filterbanks -> Slashes memory and battery by 25% compared to LSTM -> Triggers main CPU only when wake-word ('Hey Google') is detected.

Industrial Impact & Benefit: Maintains 98.7% wake-word detection accuracy while drawing less than 15 milli-Watts on battery power.

Key Oral Viva Voce Checkpoints

Q1: Who invented the Gated Recurrent Unit (GRU) and in what year?

Ans: Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio in 2014.

Q2: What are the two gates in a GRU?

Ans: The Reset Gate and the Update Gate.

Q3: What is the function of the Reset Gate in a GRU?

Ans: It determines how much of the previous hidden state h^(t-1) should be ignored when computing the new candidate hidden state h_tilde^(t).

Q4: What is the function of the Update Gate in a GRU?

Ans: It controls the linear interpolation balance between the previous hidden state h^(t-1) and the candidate hidden state h_tilde^(t).

Q5: How does GRU combine the functions of the LSTM Forget and Input gates?

Ans: It couples them into a single update gate z: the old state is scaled by (1 - z) while the candidate state is scaled by z.